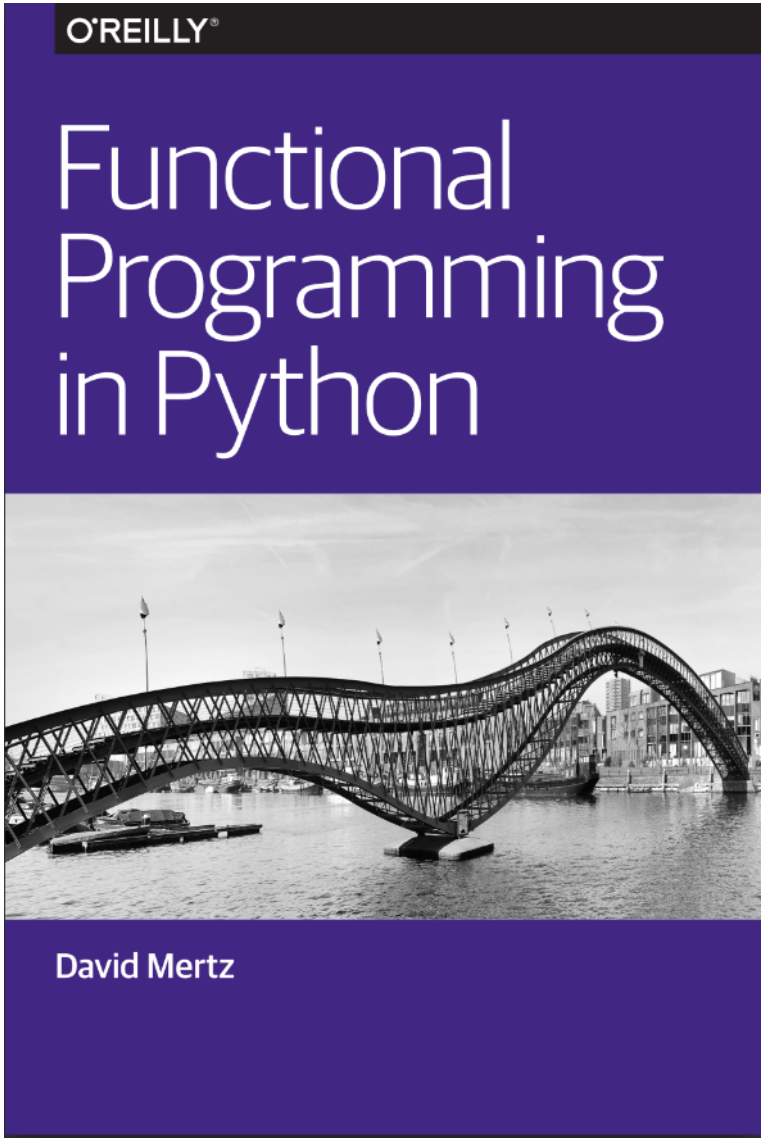
Functional Programming in Python

Jakub Dajczak

Gdańsk University of Technology

Gdańsk, 2025

Based on



Functional Programming in Python

Table of Contents

Hints

Type hints improve code readability and help with static type checking.

```
>>> from typing import List, Dict, Tuple

>>> hello : str = "Hello, World!"
>>> names : List[str] = ["Alice", "Bob"]
>>> scores : Dict[str, int] = {"Alice": 100, "Bob": 95}
>>> vector : Tuple[int, int, int] = (1, 2, 3)
```

They can be also used in function definitions:

```
>>> def say_hello(names: List[str]) -> int:
...     for name in names:
...         print(f"Hello, {name}!")
...     return len(names)
```

Hints are stored in the `__annotations__` attribute of the function.

```
>>> say_hello.__annotations__
{'names': List[str], 'return': int}
```

Type Aliases

Type aliases can make complex type hints more readable and easier to manage. Using the `TypeAlias` from the `typing` module:

```
>>> from typing import TypeAlias, Tuple

>>> Vector3D : TypeAlias = Tuple[float, float, float]
```

Alternatively, using the `type` keyword:

```
>>> type Vector3D = Tuple[float, float, float]
```

The `Vector3D` alias represents a 3D vector as a tuple of three floats.

```
>>> v1 : Vector3D = (1.0, 2.4, 3.0)
>>> v2 : Vector3D = (4.0, 5.0, 6.1)
>>> add(v1, v2)
(5.0, 7.4, 9.1)
```

namedtuple

collections module provides the namedtuple function to create simple, immutable record types.

```
>>> from collections import namedtuple
>>> Point = namedtuple('Point', ['x', 'y'])
>>> p = Point(1, 2)
```

Instances of Point have named fields and can be accessed by index or attribute.

```
>>> p.x
1
>>> p[1]
2
>>> p
Point(x=1, y=2)
```

dataclass

dataclasses module provides a dataclass decorator to create classes with less boilerplate.

```
>>> from dataclasses import dataclass

>>> @dataclass(frozen=True)
... class Point:
...     x: int
...     y: int
```

Instances of Point are mostly immutable by setting the frozen parameter to True.

```
>>> p = Point(1, 2)
>>> p.x
1
>>> p.y
2
>>> p
Point(x=1, y=2)
```

List Comprehensions

List comprehensions provide a concise way to create lists.
Traditional `for` loop to filter and modify elements in a list:

```
>>> collection = []
>>> for datum in data_set:
...     if condition(datum):
...         collection.append(datum)
...     else:
...         new = modify(datum)
...         collection.append(new)
```

Using a list comprehension for the same task:

```
>>> collection = [datum if condition(datum) else modify(datum) for
    datum in data_set]
```

List comprehensions are more compact, readable, and often faster than traditional loops.

Other Comprehensions

Dictionary comprehension to create a mapping of indices to characters:

```
>>> {i:chr(65+i) for i in range(6)}
{0: 'A', 1: 'B', 2: 'C', 3: 'D', 4: 'E', 5: 'F'}
```

Set comprehension to create a set of characters:

```
>>> {chr(65+i) for i in range(6)}
{'A', 'B', 'C', 'D', 'E', 'F'}
```

Generator expression to lazily generate characters:

```
>>> (chr(65+i) for i in range(6))
<generator object <genexpr> at 0x7f8b1c1b3d60>
```

Generators

Generator functions declare a function that behaves like an iterator, allowing it to be used in a for loop. Unlike lists, they are lazy and do not store all the values in memory.

```
>>> def count(n):
...     for i in range(n):
...         yield i

>>> c = count(5)
>>> c
<generator object count at 0x7f8b1c1b3d60>
>>> list(c)
[0, 1, 2, 3, 4]
```

Recursion

Python supports recursion, but it is not optimized for tail calls or deep recursion. It is best suited for problems that can be solved with a small number of recursive calls.

```
>>> def factorialI(N):
...     product = 1
...     while N >= 1:
...         product *= N
...         N -= 1
...     return product


>>> def factorialR(N):
...     return 1 if N <= 1 else N * factorialR(N-1)
```

Pure Functions

Pure functions are functions where the output value is determined only by its input values, without observable side effects.
An impure function example:

```
>>> def print_and_add(x, y):
...     print(f"Adding {x} and {y}")
...     return x + y

>>> print_and_add(5, 3)
Adding 5 and 3
8
```

The function `print_and_add` is impure because it has a side effect (printing).

```
>>> def add(x, y):
...     return x + y

>>> add(5, 3)
8
```

Lambda Functions

Lambda functions are small anonymous functions defined with the `lambda` keyword and they can be used wherever function objects are required.

```
>>> double = lambda x: x * 2
>>> double(3)
6
```

The name of a lambda function can be accessed or set using the `__qualname__` attribute.

```
>>> double.__qualname__
'lambda'
>>> len.__qualname__
'len'
```

Closures

Python supports closures - functions that refer to variables from the scope in which they were defined.

```
>>> def f(x):
...     def g(y):
...         return x + y
...     return g

>>> add3 = f(3)
>>> add3(3)
6
>>> add3(5)
8
```

Unexpected Behavior

Closures can lead to unexpected behavior when the enclosed variable is mutable.

```
>>> adders = []
>>> for n in range(5):
...     adders.append(lambda m: m+n)
>>> [adder(10) for adder in adders]
[14, 14, 14, 14, 14]
>>> n = 10
>>> [adder(10) for adder in adders]
[20, 20, 20, 20, 20]
```

Unexpected Behavior

Fortunately, a small change brings behavior that probably better meets our goal:

```
>>> adders = []
>>> for n in range(5):
...     adders.append(lambda m, n=n: m+n)
>>> [adder(10) for adder in adders]
[10, 11, 12, 13, 14]
>>> n = 10
>>> [adder(10) for adder in adders]
[10, 11, 12, 13, 14]
>>> add4 = adders[4]
>>> add4(10, 100)  # Can override the bound value
110
```

Callable Instances

Callable instances can be created in Python by defining the `__call__` method in a class. This approach is useful for creating objects that behave like functions.

```
>>> class Adder:
...     def __init__(self, x):
...         self.__x = x
...     def __call__(self, y):
...         return self.__x + y
```

`Adder` instances can be called with an argument to add a predefined value.

```
>>> add5 = Adder(5)
>>> add5(10)
15
>>> add5(3)
8
```

match Statement

```
>>> from dataclasses import dataclass
>>> from typing import Union
```

Define dataclasses for shapes:

```
>>> @dataclass
... class Circle:
...     radius: float

>>> @dataclass
... class Rectangle:
...     width: float
...     height: float

>>> @dataclass
... class Triangle:
...     base: float
...     height: float
```

A union type to represent all shapes:

```
>>> Shape = Union[Circle, Rectangle, Triangle]
```

match Statement

match can be used to define a function that describes a shape:

```
>>> def describe_shape(shape: Shape) -> str:
...     match shape:
...         case Circle(radius=r):
...             return f"A circle with radius {r}."
...         case Rectangle(width=w, height=h):
...             return f"A rectangle with width {w} and height {h}.
    "
...         case Triangle(base=b, height=h):
...             return f"A triangle with base {b} and height {h}."
...         case _:
...             return "Not a valid shape."

>>> describe_shape(Circle(5))
'A circle with radius 5.'
>>> describe_shape(Rectangle(10, 20))
'A rectangle with width 10 and height 20.'
>>> describe_shape(Triangle(8, 6))
'A triangle with base 8 and height 6.'
>>> describe_shape(5)
'Not a valid shape.'
```

match Statement for Lists

match statement can also be used to define a function that recursively sums a list of numbers:

```
>>> def sum_list(numbers):
...     match numbers:
...         case []:
...             return 0
...         case [head, *tail]:
...             return head + sum_list(tail)

>>> numbers = [1, 2, 3, 4, 5]
>>> sum_list(numbers)
15
>>> sum_list([])
0
```

Higher-Order Functions

In Python, functions are first-class objects, allowing them to be passed as arguments, returned from other functions, and assigned to variables.

```
>>> def shout(text):
...     return text.upper()
...
>>> def whisper(text):
...     return text.lower()
...
>>> def greet(func):
...     return func("Hi, I am created by a function passed as an
    argument.")

>>> greet(shout)
'HI, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT.'
>>> greet(whisper)
'hi, i am created by a function passed as an argument.'
```

Function Composition

Using higher-order functions, it is possible to compose functions by chaining them together and creating a simple pipeline.

```
>>> def compose(*funcs):
...     def inner(data, funcs=funcs):
...         result = data
...         for f in reversed(funcs):
...             result = f(result)
...         return result
...     return inner

>>> times2 = lambda x: x*2
>>> minus3 = lambda x: x-3
>>> mod6 = lambda x: x%6
>>> f = compose(mod6, times2, minus3)
>>> all(f(i)==((i-3)*2)%6 for i in range(1000000))
True
```

Decorators

Decorators provide a convenient way to wrap additional behavior around function calls, allowing reusable modifications like logging or validation.

```
>>> def decorator(func):
...     def wrapper():
...         print("Before calling the function.")
...         func()
...         print("After calling the function.")
...     return wrapper

>>> @decorator
... def greet():
...     print("Hello, World!")

>>> greet()
Before calling the function.
Hello, World!
After calling the function.
```

The operator Module

This module provides function equivalents of Python's built-in operators, making it easier to pass them around or compose when building transformations alongside other utilities.

```
>>> import operator

>>> operator.add(1, 2) == 1 + 2
True
>>> operator.mul(3, 4) == 3 * 4
True
>>> operator.pow(2, 3) == 2 ** 3
True
```

The itertools Module

The itertools module provides iterable utilities such as accumulate for partial computations or repeated applications of a function.

```
>>> from operator import mul
>>> from itertools import accumulate, repeat
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]

>>> list(accumulate(data, max))
[3, 4, 6, 6, 6, 9, 9, 9, 9, 9]

>>> list(accumulate(data, mul))
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

>>> update = lambda balance, payment: round(balance * 1.05) -
    payment
>>> list(accumulate(repeat(90, 10), update, initial=1000))
[1000, 960, 918, 874, 828, 779, 728, 674, 618, 559, 497]
```

The functools Module

The functools module provides higher-order functions that operate on other functions, making it easier to compose or partially apply functions.

```
>>> from functools import reduce
>>> from operator import mul

>>> def factorial(n):
...     return reduce(mul, range(1, n+1), 1)


>>> from functools import partial

>>> def multiply(x, y):
...     return x * y

>>> double = partial(multiply, 2)
>>> double(4)
8
```

About returns

Make your functions return something meaningful, typed, and safe!

`https://github.com/dry-python/returns`

Maybe

```
>>> @dataclass
... class Address(object):
...     street: Optional[str]

>>> @dataclass
... class User(object):
...     address: Optional[Address]

>>> @dataclass
... class Order(object):
...     user: Optional[User]

>>> def get_street_address(order: Order) -> Maybe[str]:
...     return Maybe.from_optional(order.user).bind_optional(
...         lambda user: user.address,
...     ).bind_optional(
...         lambda address: address.street,
...     )
```

Maybe

```
>>> with_address = Order(User(Address('Some street')))
>>> empty_user = Order(None)
>>> empty_address = Order(User(None))
>>> empty_street = Order(User(Address(None)))

>>> get_street_address(with_address)
<Some: Some street>

>>> get_street_address(empty_user)
<Nothing>

>>> get_street_address(empty_address)
<Nothing>

>>> get_street_address(empty_street)
<Nothing>
```

Maybe Decorator

```
>>> from returns.maybe import maybe

>>> @maybe
... def number(num: int) -> Optional[int]:
...     if num > 0:
...         return num
...     return None

>>> number(1)
<Some: 1>
>>> number(-1)
<Nothing>
```

Result

```
>>> from returns.result import Result, Success, Failure

>>> def find_user(user_id: int) -> Result['User', str]:
...     user = User.objects.filter(id=user_id)
...     if user.exists():
...         return Success(user[0])
...     return Failure('User was not found')

>>> find_user(1)
<Success(User{id: 1, ...})>

>>> find_user(0)
<Failure('User was not found')>
```

Result Decorator

```
>>> from returns.result import safe

>>> @safe  # Will convert type to: Callable[[int], Result[float,
    Exception]]
... def divide(number: int) -> float:
...     return number / number

>>> divide(1)
<Success(1.0)>

>>> divide(0)
<Failure: division by zero>
```

Pointfree

```
>>> from returns.pointfree import map_

>>> def as_int(arg: str) -> int:
...     return ord(arg)

>>> container : Maybe[str] = Some('a')
>>> map_(as_int)(container) == container.map(as_int)
True
```

Trampolines

```
>>> from typing import Union, List
>>> from returns.trampolines import Trampoline, trampoline

>>> @trampoline
... def accumulate(numbers: List[int], acc: int = 0) -> Union[int,
    Trampoline[int]]:
...     if not numbers:
...         return acc
...     number = numbers.pop()
...     return Trampoline(accumulate, numbers, acc + number)

>>> accumulate([1, 2])
3
>>> accumulate([1, 2, 3])
6
```

Fold

```
>>> from typing import Callable
>>> from returns.maybe import Some
>>> from returns.iterables import Fold

>>> def sum_two(first: int) -> Callable[[int], int]:
...     return lambda second: first + second

>>> Fold.loop(
...     [Some(1), Some(2), Some(3)],
...     Some(10),
...     sum_two,
... )
<Some: 16>
```

Pipelines

```
>>> from returns.result import Result, Success, Failure

>>> def regular_function(arg: int) -> float:
...     return float(arg)

>>> def returns_container(arg: float) -> Result[str, ValueError]:
...     if arg != 0:
...         return Success(str(arg))
...     return Failure(ValueError('Wrong arg'))

>>> def also_returns_container(arg: str) -> Result[str, ValueError
    ]:
...     return Success(arg + '!')
```

flow function

```
>>> from returns.pipeline import flow
>>> from returns.pointfree import bind

>>> flow(
...     1,
...     regular_function,
...     returns_container,
...     bind(also_returns_container),
... )
<Success: '1.0!'>

>>> flow(
...     0,
...     regular_function,
...     returns_container,
...     bind(also_returns_container),
... )
<Failure: Wrong arg>
```

pipe function

```
>>> from returns.pipeline import pipe
>>> from returns.pointfree import bind

>>> transaction = pipe(
...     regular_function,
...     returns_container,
...     bind(also_returns_container),
... )
>>> transaction(1)
<Success: '1.0!'>
>>> transaction(0)
<Failure: Wrong arg>
```

Mappable

From the `returns` documentation:
Something is considered mappable if we can map it using a function, generally map is a method that accepts a function.

```
>>> from returns.maybe import Maybe, Some

>>> def can_be_mapped(string: str) -> str:
...     return string + '!'

>>> maybe_str: Maybe[str] = Some('example')

>>> assert maybe_str.map(can_be_mapped) == Some('example!')
```

Applicative

From the `returns` documentation:
Something is considered applicative if it is a functor already and, moreover, we can apply another container to it and construct a new value with `.from_value` method.

```
>>> from returns.maybe import Maybe, Some

>>> maybe_str = Maybe.from_value('example')

>>> maybe_func = Maybe.from_value(len)  # we use function as a
    value!

>>> assert maybe_str.apply(maybe_func) == Some(7)
```

Bindable

From the `returns` documentation:
Bindable is something that we can bind with a function.

```
>>> from returns.maybe import Nothing, Maybe, Some

>>> def bindable(string: str) -> Maybe[str]:
...     return Some(string + 'b')

>>> assert Some('a').bind(bindable) == Some('ab')
>>> assert Nothing.bind(bindable) == Nothing
```

Bibliography I

- https://archive.org/details/functional-programming-python/mode/2up
- https://docs.python.org/3/howto/functional.html
- https://docs.python.org/3/library/typing.html
- https://github.com/sfermigier/awesome-functional-python?tab=readme-ov-file
- https://github.com/dry-python/returns
- https://docs.python.org/3/library/operator.html
- https://github.com/vickumar1981/functional_python/blob/master/functional-python.pdf
- https://noklam-data.medium.com/how-to-achieve-partial-immutability-with-python-dataclasses-or-
- https://docs.python.org/3/library/itertools.html
- https://www.programiz.com/python-programming/closure
- https://peps.python.org/pep-0636/

Bibliography II

- https://www.geeksforgeeks.org/python-match-case-statement/
- https://www.geeksforgeeks.org/python-lambda-anonymous-functions-filter-map-reduce/
- https://enauczanie.pg.edu.pl/moodle/course/view.php?id=41790
- https://www.geeksforgeeks.org/namedtuple-in-python/
- https://wiki.python.org/moin/Generators

Thank you!

Questions?